

Multimethods

Clojure multimethods provide a flexible way to associate a function with a set of inputs. This is similar to Java polymorphism but more general. When you call a Java method, Java selects a specific implementation to execute by examining the *type* of a *single object*. When you call a Clojure multimethod, Clojure selects a specific implementation to execute by examining the result of *any function you choose*, applied to *all* the function's arguments.

In this chapter, you will develop a thirst for multimethods by first living without them. Then you will build an increasingly complex series of multimethod implementations, first using multimethods to simulate polymorphism and then using multimethods to implement various ad hoc taxonomies.

Multimethods in Clojure are used much less often than polymorphism in object-oriented languages. But where they are used, they are often the key feature in the code. [Section 8.5, When Should I Use Multimethods?, on page 198](#) explores how multimethods are used in several open source Clojure projects and offers guidelines for when to use them in your own programs.

If you are reading the book in chapter order, then once you have completed this chapter, you will have seen all the key features of the Clojure language.

8.1 Living Without Multimethods

The best way to appreciate multimethods is to spend a few minutes living without them, so let's do that. Clojure can already print anything with `print/println`. But pretend for a moment that these functions do not exist and that you need to build a generic print mechanism. To get started, create a my-print function that can print a string to the standard output stream `*out*`:

```
src/examples/life_without_multi.clj
(defn my-print [ob]
  (.write *out* ob))
```

Next, create a my-println that simply calls my-print and then adds a line feed:

```
src/examples/life_without_multi.clj
(defn my-println [ob]
  (my-print ob)
  (.write *out* "\n"))
```

The line feed makes my-println's output easier to read when testing at the REPL. For the remainder of this section, you will make changes to my-print and test them by calling my-println. Test that my-println works with strings:

```
(my-println "hello")
| hello
-> nil
```

That is nice, but my-println does not work quite so well with nonstrings such as nil:

```
(my-println nil)
-> java.lang.NullPointerException
```

That's not a big deal, though. Just use cond to add special-case handling for nil:

```
src/examples/life_without_multi.clj
(defn my-print [ob]
  (cond
    (nil? ob) (.write *out* "nil")
    (string? ob) (.write *out* ob)))
```

With the conditional in place, you can print nil with no trouble:

```
(my-println nil)
| nil
-> nil
```

Of course, there are still all kinds of types that my-println cannot deal with. If you try to print a vector, neither of the cond clauses will match, and the program will print nothing at all:

```
(my-println [1 2 3])
-> nil
```

By now you know the drill. Just add another cond clause for the vector case. The implementation here is a little more complex, so you might want to separate the actual printing into a helper function, such as my-print-vector:

```
src/examples/life_without_multi.clj
(require '[clojure.string :as str])
(defn my-print-vector [ob]
  (.write *out* "[")
  (.write *out* (str/join " " ob))
  (.write *out* "]"))

(defn my-print [ob]
  (cond
    (vector? ob) (my-print-vector ob)
    (nil? ob) (.write *out* "nil")
    (string? ob) (.write *out* ob)))
```

Make sure that you can now print a vector:

```
(my-println [1 2 3])
| [1 2 3]
-> nil
```

my-println now supports three types: strings, vectors, and nil. And you have a road map for new types: just add new clauses to the cond in my-println. But it is a crummy road map, because it conflates two things: the decision process for selecting an implementation and the specific implementation detail.

You can improve the situation somewhat by pulling out helper functions like my-print-vector. However, then you have to make two separate changes every time you want to add new feature to my-println:

- Create a new type-specific helper function.
- Modify the existing my-println to add a new cond invoking the feature-specific helper.

What you really want is a way to add new features to the system by adding new code in a single place, without having to modify any existing code. Clojure offers this by way of protocols, covered in [Section 6.3, Protocols, on page 147](#), and multimethods.

8.2 Defining Multimethods

To define a multimethod, use defmulti:

```
(defmulti name dispatch-fn)
```

name is the name of the new multimethod, and Clojure will invoke dispatch-fn against the method arguments to select one particular method (implementation) of the multimethod.

Consider my-print from the previous section. It takes a single argument, the thing to be printed, and you want to select a specific implementation based on the type of that argument. So, dispatch-fn needs to be a function of one argument that returns the type of that argument. Clojure has a built-in function matching this description, namely, class. Use class to create a multimethod called my-print:

```
src/examples/multimethods.clj
(defmulti my-print class)
```

At this point, you have provided a description of how the multimethod will select a specific method but no actual specific methods. Unsurprisingly, attempts to call my-print will fail:

```
(my-println "foo")
-> java.lang.IllegalArgumentException: \
No method for dispatch value
```

To add a specific method implementation to my-println, use defmethod:

```
(defmethod name dispatch-val & fn-tail)
```

name is the name of the multimethod to which an implementation belongs. Clojure matches the result of defmulti's dispatch function with dispatch-val to select a method, and fn-tail contains arguments and body forms just like a normal function.

Create a my-print implementation that matches on strings:

```
src/examples/multimethods.clj
(defmethod my-print String [s]
  (.write *out* s))
```

Now, call my-println with a string argument:

```
(my-println "stu")
| stu
-> nil
```

Next, create a my-print that matches on nil:

```
src/examples/multimethods.clj
(defmethod my-print nil [s]
  (.write *out* "nil"))
```

Notice that you have solved the problem raised in the previous section. Instead of being joined in a big cond, each implementation of my-println is separate. Methods of a multimethod can live anywhere in your source, and you can add new ones any time, without having to touch the original code.

Dispatch Is Inheritance-Aware

Multimethod dispatch knows about Java inheritance. To see this, create a `my-print` that handles `Number` by printing a number's `toString` representation:

```
src/examples/multimethods.clj
(defmethod my-print Number [n]
  (.write *out* (.toString n)))
```

Test the `Number` implementation with an integer:

```
(my-println 42)
| 42
-> nil
```

42 is an `Integer`, not a `Number`. Multimethod dispatch is smart enough to know that an integer is a number and match anyway. Internally, dispatch uses the `isa?` function:

```
(isa? child parent)
```

`isa?` knows about Java inheritance, so it knows that an `Integer` is a `Number`:

```
(isa? Integer Number)
-> true
```

`isa?` is not limited to inheritance. Its behavior can be extended dynamically at runtime, as you will see later in [Section 8.4, Creating Ad Hoc Taxonomies, on page 194](#).

Multimethod Defaults

It would be nice if `my-print` could have a fallback representation that you could use for any type you have not specifically defined. You can use `:default` as a dispatch value to handle any methods that do not match anything more specific. Using `:default`, create a `my-println` that prints the Java `toString` value of objects, wrapped in `#<>`:

```
src/examples/multimethods.clj
(defmethod my-print :default [s]
  (.write *out* "#<")
  (.write *out* (.toString s))
  (.write *out* ">"))
```

Now test that `my-println` prints random things, using the default method:

```
(my-println (java.sql.Date. 0))
-> #<1969-12-31>

(my-println (java.util.Random.))
-> #<java.util.Random@1c398896>
```

In the unlikely event that `:default` already has some specific meaning in your domain, you can create a multimethod using this alternate signature:

```
(defmulti name dispatch-fn :default default-value)
```

The `default-value` lets you specify your own default. Maybe you would like to call it `:everything-else`:

```
src/examples/multimethods/default.clj
```

```
(defmulti my-print class :default :everything-else)
(defmethod my-print String [s]
  (.write *out* s))
(defmethod my-print :everything-else [_]
  (.write *out* "Not implemented yet..."))
```

Any dispatch value that does not otherwise match will now match against `:everything-else`.

Dispatching a multimethod on the type of the first argument, as you have done with `my-print`, is by far the most common kind of dispatch. In many object-oriented languages, in fact, it is the *only* kind of dynamic dispatch, and it goes by the name *polymorphism*.

Clojure's dispatch is much more general. Let's add a few complexities to `my-print` and move beyond what is possible with plain ol' polymorphism.

8.3 Moving Beyond Simple Dispatch

Clojure's `print` function prints various “sequencey” things as lists. If you wanted `my-print` to do something similar, you could add a method that dispatched on a collection interface high in the Java inheritance hierarchy, such as `Collection`:

```
src/examples/multimethods.clj
```

```
(require '[clojure.string :as str])
(defmethod my-print java.util.Collection [c]
  (.write *out* "(")
  (.write *out* (str/join " " c))
  (.write *out* "))")
```

Now, try various sequences to see that they get a nice print representation:

```
(my-println (take 6 (cycle [1 2 3])))
| (1 2 3 1 2 3)
-> nil
```

```
(my-println [1 2 3])
| (1 2 3)
-> nil
```

Perfectionist that you are, you cannot stand that vectors print with rounded braces, unlike their literal square-brace syntax. So, add yet another `my-print` method, this time to handle vectors. Vectors all implement an `IPersistentVector`, so this should work:

`src/examples/multimethods.clj`

```
(defmethod my-print clojure.lang.IPersistentVector [c]
  (.write *out* "[" )
  (.write *out* (str/join " " c))
  (.write *out* "]"))
```

But it doesn't work. Instead, printing vectors now throws an exception:

```
(my-println [1 2 3])
-> java.lang.IllegalArgumentException: Multiple methods match
dispatch value: class clojure.lang.LazilyPersistentVector ->
interface clojure.lang.IPersistentVector and
interface java.util.Collection,
and neither is preferred
```

The problem is that two dispatch values now match for vectors: `Collection` and `IPersistentVector`. Many languages constrain method dispatch to make sure these conflicts never happen, such as by forbidding multiple inheritance. Clojure takes a different approach. You can create conflicts, and you can resolve them with `prefer-method`:

`(prefer-method multi-name loved-dispatch dissed-dispatch)`

When you call `prefer-method` for a multimethod, you tell it to prefer the `loved-dispatch` value over the `dissed-dispatch` value whenever there is a conflict. Since you want the vector version of `my-print` to trump the collection version, tell the multimethod what you want:

`src/examples/multimethods.clj`

```
(prefer-method
  my-print clojure.lang.IPersistentVector java.util.Collection)
```

Now, you should be able to route both vectors and other sequences to the correct method implementation:

```
(my-println (take 6 (cycle [1 2 3])))
| (1 2 3 1 2 3)
-> nil
```

```
(my-println [1 2 3])
| [1 2 3]
-> nil
```

Many languages create complex rules, or arbitrary limitations, in order to resolve ambiguities in their systems for dispatching functions. Clojure allows

a much simpler approach: just don't worry about it! If there is an ambiguity, use `prefer-method` to resolve it.

8.4 Creating Ad Hoc Taxonomies

Multimethods let you create ad hoc taxonomies, which can be helpful when you discover type relationships that are not explicitly declared as such.

For example, consider a financial application that deals with checking and savings accounts. Define a Clojure struct for an account, using a tag to distinguish the two. Place the code in the namespace `examples.multimethods.account`. To do this, you will need to create a file named `examples/multimethods/account.clj` on your classpath¹ and then enter the following code:

```
src/examples/multimethods/account.clj
(ns examples.multimethods.account)

(defstruct account :id :tag :balance)
```

Now, you are going to create two different checking accounts, tagged as `::Checking` and `::Savings`. The capital names are a Clojure convention to show the keywords are acting as types. The doubled `::` causes the keywords to resolve in the current namespace. To see the namespace resolution happen, compare entering `:Checking` and `::Checking` at the REPL:

```
:Checking
-> :Checking

::Checking
-> :user/Checking
```

Placing keywords in a namespace helps prevent name collisions with other people's code. When you want to use `::Savings` or `::Checking` from another namespace, you will need to fully qualify them:

```
(struct account 1 ::examples.multimethods.account/Savings 100M)
-> {:id 1, :tag examples.multimethods.account/Savings, :balance 100M}
```

Full names get tedious quickly, so you can use alias to specify a shorter alias for a long namespace name:

```
(alias short-name-symbol namespace-symbol)
```

Use alias to create the short name `acc`:

1. Note that the example code for the book includes a completed version of this example, already on the classpath. To work through the example yourself, simply move or rename the completed example to get it out of the way.

```
(alias 'acc 'examples.multimethods.account)
-> nil
```

Now that the `acc` alias is available, create two top-level test objects, a savings account and a checking account:

```
(def test-savings (struct account 1 ::acc/Savings 100M))
-> #'user/test-savings

(def test-checking (struct account 2 ::acc/Checking 250M))
-> #'user/test-checking
```

Note that the trailing `M` creates a `BigDecimal` literal and does not mean you have millions of dollars.

The interest rate for checking accounts is 0 and for savings accounts is 5 percent. Create a multimethod `interest-rate` that dispatches based on `:tag`, like so:

```
src/examples/multimethods/account.clj
(defmulti interest-rate :tag)
(defmethod interest-rate ::acc/Checking [_] 0M)
(defmethod interest-rate ::acc/Savings [_] 0.05M)
```

Check your `test-savings` and `test-checking` to make sure that `interest-rate` works as expected.

```
(interest-rate test-savings)
-> 0.05M

(interest-rate test-checking)
-> 0M
```

Accounts have an annual service charge, with rules as follows:

- Normal checking accounts pay a \$25 service charge.
- Normal savings accounts pay a \$10 service charge.
- Premium accounts have no fee.
- Checking accounts with a balance of \$5,000 or more are premium.
- Savings accounts with a balance of \$1,000 or more are premium.

In a realistic example, the rules would be more complex. Premium status would be driven by average balance over time, and there would probably be other ways to qualify. But the previous rules are complex enough to demonstrate the point.

You could implement service-charge with a bunch of conditional logic, but premium feels like a type, even though there is no explicit premium tag on an account. Create an account-level multimethod that returns `::Premium` or `::Basic`:

```
src/examples/multimethods/account.clj
```

```
(defmulti account-level :tag)
(defmethod account-level ::acc/Checking [acct]
  (if (>= (:balance acct) 5000) ::acc/Premium ::acc/Basic))
(defmethod account-level ::acc/Savings [acct]
  (if (>= (:balance acct) 1000) ::acc/Premium ::acc/Basic))
```

Test account-level to make sure that checking and savings accounts require different balance levels to reach ::Premium status:

```
(account-level (struct account 1 ::acc/Savings 2000M))
-> :examples.multimethods.account/Premium
```

```
(account-level (struct account 1 ::acc/Checking 2000M))
-> :examples.multimethods.account/Basic
```

Now you might be tempted to implement service-charge using account-level as a dispatch function:

```
src/examples/multimethods/service_charge_1.clj
```

```
; bad approach
(defmulti service-charge account-level)
(defmethod service-charge ::Basic [acct]
  (if (= (:tag acct) ::Checking) 25 10))
(defmethod service-charge ::Premium [_] 0)
```

The conditional logic in service-charge for ::Basic is exactly the kind of type-driven conditional that multimethods should help us avoid. The problem here is that you are already dispatching by account-level, and now you need to be dispatching by :tag as well. No problem—you can dispatch on *both*. Write a service-charge whose dispatch function calls both account-level and :tag, returning the results in a vector:

```
src/examples/multimethods/service_charge_2.clj
```

```
(defmulti service-charge (fn [acct] [(account-level acct) (:tag acct)]))
(defmethod service-charge [::acc/Basic ::acc/Checking] [_] 25)
(defmethod service-charge [::acc/Basic ::acc/Savings] [_] 10)
(defmethod service-charge [::acc/Premium ::acc/Checking] [_] 0)
(defmethod service-charge [::acc/Premium ::acc/Savings] [_] 0)
```

This version of service-charge dispatches against two different taxonomies: the :tag intrinsic to an account and the externally defined account-level. Try a few accounts to verify that service-charge works as expected:

```
(service-charge {:tag ::acc/Checking :balance 1000})
-> 25
```

```
(service-charge {:tag ::acc/Savings :balance 1000})
-> 0
```

Notice that the previous tests did not even bother to create a “real” account for testing. Structs like `account` are simply maps that are optimized for storing particular fields, but nothing is stopping you from using a plain old map if you find it more convenient.

Adding Inheritance to Ad Hoc Types

There is one further improvement you can make to `service-charge`. Since all premium accounts have the same service charge, it feels redundant to have to define two separate `service-charge` methods for `::Savings` and `::Checking` accounts. It would be nice to have a parent type `::Account` so you could define a multi-method that matches `::Premium` for any kind of `::Account`. Clojure lets you define arbitrary parent-child relationships with `derive`:

```
(derive child parent)
```

Using `derive`, you can specify that both `::Savings` and `::Checking` are kinds of `::Account`:

```
src/examples/multimethods/service_charge_3.clj
(derive ::acc/Savings ::acc/Account)
(derive ::acc/Checking ::acc/Account)
```

When you start to use `derive`, `isa?` comes into its own. In addition to understanding Java inheritance, `isa?` knows all about derived relationships:

```
(isa? ::acc/Savings ::acc/Account)
-> true
```

Now that Clojure knows that `Savings` and `Checking` are `Accounts`, you can define a `service-charge` using a single method to handle `::Premium`:

```
src/examples/multimethods/service_charge_3.clj
(defmulti service-charge (fn [acct] [(account-level acct) (:tag acct)]))
(defmethod service-charge [::acc/Basic ::acc/Checking] [_] 25)
(defmethod service-charge [::acc/Basic ::acc/Savings] [_] 10)
(defmethod service-charge [::acc/Premium ::acc/Account] [_] 0)
```

At first glance, you may think that `derive` and `isa?` simply duplicate functionality that is already available to Clojure via Java inheritance. This is not the case. Java inheritance relationships are forever fixed at the moment you define a class. derived relationships can be created when you need them and can be *applied to existing objects without their knowledge or consent*. So, when you discover a useful relationship between existing objects, you can derive that relationship without touching the original objects’ source code and without creating tiresome “wrapper” classes.

If the number of different ways you might define a multimethod has your head spinning, don't worry. In practice, most Clojure code uses multimethods sparingly. Let's take a look at some open source Clojure code to get a better idea of how multimethods are used.

8.5 When Should I Use Multimethods?

Multimethods are extremely flexible, and with that flexibility comes choices. How should you choose when to use multimethods, as opposed to some other technique? We approached this question from two directions, asking the following:

- Where do Clojure projects use multimethods?
- Where do Clojure projects *eschew* multimethods?

The most striking thing is that multimethods are *rare*—about one per 1,000 lines of code. So, don't worry that you are missing something important if you build a Clojure application with few, or no, multimethods. A Clojure program that defines no multimethods is not nearly as odd as an object-oriented program with no polymorphism.

Many multimethods dispatch on class. Dispatch-by-class is the easiest kind of dispatch to understand and implement. We already covered it in detail with the my-print example, so I will say no more about it here.

Clojure multimethods that dispatch on something other than class are fairly rare. We can look directly in Clojure for some examples. The `clojure.inspector` and `clojure.test` libraries use unusual dispatch functions.

The Inspector

Clojure's inspector library uses Swing to create simple views of data. You can use it to get a tree view of your system properties:

```
(use '[clojure.inspector :only (inspect inspect-tree)])
(inspect-tree (System/getProperties))
-> #<JFrame ...>
```

`inspect-tree` returns (and displays) a `JFrame` with a tree view of anything that is treeish. So, you could also pass a nested map to `inspect-tree`:

```
(inspect-tree {:clojure {:creator "Rich" :runs-on-jvm true}})
-> #<JFrame ...>
```

Treeish things are made up of nodes that can answer two questions:

- Who are my children?
- Am I a leaf node?

The treeish concepts of “tree,” “node,” and “leaf” all sound like candidates for classes or interfaces in an object-oriented design. But the inspector does not work this way. Instead, it adds a “treeish” type system in an ad hoc way to existing types, using a dispatch function named `collection-tag`:

```
; from Clojure's clojure/inspector.clj
(defn collection-tag [x]
  (cond
    (instance? java.util.Map$Entry x) :entry
    (instance? clojure.lang.IPersistentMap x) :map
    (instance? java.util.Map x) :map
    (instance? clojure.lang.Sequential x) :seq
    :else :atom))
```

`collection-tag` returns one of the keywords `:entry`, `:map`, `:seq`, or `:atom`. These act as the type system for the treeish world. The `collection-tag` function is then used to dispatch three different multimethods that select specific implementations based on the treeish type system.

```
(defmulti is-leaf collection-tag)

(defmulti get-child
  (fn [parent index] (collection-tag parent)))

(defmulti get-child-count collection-tag)
; method implementations elided for brevity
```

The treeish type system is added around the existing Java type system. Existing objects do not have to *do* anything to become treeish; the inspector library does it for them. Treeish demonstrates a powerful style of reuse. You can discover new type relationships in existing code and take advantage of these relationships simply, without having to modify the original code.

clojure.test

The `clojure.test` library in Clojure lets you write several different kinds of assertions using the `is` macro. You can assert that arbitrary functions are true. For example, 10 is not a string:

```
(use :reload '[clojure.test :only (is)])
(is (string? 10))
```

```
FAIL in clojure.lang.PersistentList$EmptyList@1 (NO_SOURCE_FILE:2)
expected: (string? 10)
actual: (not (string? 10))
-> false
```

Although you can use an arbitrary function, `is` knows about a few specific functions and provides more detailed error messages. For example, you can check that a string is not an instance? of `Collection`:

```
(is (instance? java.util.Collection "foo"))
```

```
FAIL in clojure.lang.PersistentList$EmptyList@1 (NO_SOURCE_FILE:3)
expected: (instance? java.util.Collection "foo")
actual: java.lang.String
-> false
```

`is` also knows about `=`. Verify that power does not equal wisdom.

```
(is (= "power" "wisdom"))
```

```
FAIL in clojure.lang.PersistentList$EmptyList@1 (NO_SOURCE_FILE:4)
expected: (= "power" "wisdom")
actual: (not (= "power" "wisdom"))
-> false
```

Internally, `is` uses a multimethod named `assert-expr`, which dispatches not on the type but on the actual *identity* of its first argument:

```
(defmulti assert-expr (fn [form message] (first form)))
```

Since the first argument is a symbol representing what function to check, this amounts to yet another ad hoc type system. This time, there are three types: `=`, `instance?`, and everything else.

The various `assert-expr` methods add specific error messages associated with different functions you might call from `is`. Because multimethods are open ended, you can add your own `assert-expr` methods with improved error messages for other functions you frequently pass to `is`.

Counterexamples

As you saw in [Section 8.4, *Creating Ad Hoc Taxonomies*, on page 194](#), you can often use multimethods to hoist branches that are based on type out of the main flow of your functions. To find counterexamples where multimethods should not be used, we looked through Clojure's core to find type branches that had *not* been hoisted to multimethods.

A simple example is Clojure's `class`, which is a null-safe wrapper for the underlying Java `getClass`. Minus comments and metadata, `class` is as follows:

```
(defn class [x]
  (if (nil? x) x (. x (getClass))))
```

You could write your own version of class as a multimethod by dispatching on identity:

```
src/examples/multimethods.clj
(defmulti my-class identity)
(defmethod my-class nil [_] nil)
(defmethod my-class :default [x] (.getClass x))
```

Any nil-check could be rewritten this way. But I find the original class function easier to read than the multimethod version. This is a nice “exception that proves the rule.” Even though class branches on type, the branching version is easier to read.

Use the following general rules when deciding whether to create a function or a multimethod:

- If a function branches based on a type, or multiple types, consider a multimethod.
- Types are whatever you discover them to be. They do not have to be explicit Java classes or data tags.
- You should be able to interpret the dispatch value of a defmethod without having to refer to the defmulti.
- Do not use multimethods merely to handle optional arguments or recursion.

When in doubt, try writing the function in both styles, and pick the one that seems more readable.

8.6 Wrapping Up

Multimethods support arbitrary dispatch. Usually multimethods work based on type relationships. Sometimes these types are formal, as in Java classes. Other times they are informal and ad hoc and emerge from the properties of objects in the system.